SafeStreet

AI-Powered Community Incident Reporting Platform

Deliverable 1 — System Design Document

Web Technologies — Course Project · Intermediate Web Technologies, Full-Stack Track

1. System Overview

SafeStreet is a full-stack incident reporting platform for municipal infrastructure issues (potholes, streetlight outages, illegal dumping, drainage, etc.). It supports four user roles — Citizen, Field Worker, Supervisor, and Admin — across a React (Vite + TypeScript) frontend and a NestJS backend, with PostgreSQL (via Prisma ORM) for persistence, Socket.io for real-time updates, and Google Gemini for AI-assisted classification, duplicate detection, response drafting, and hotspot prediction.

This document covers the four required system-design artifacts: the entity-relationship model, the full REST + WebSocket API contract, the React component tree with role-based visibility, and the AI integration architecture describing prompt structure, context management, and output handling.

2. Entity-Relationship Diagram

The schema is defined in prisma/schema.prisma and consists of five entities. Foreign keys are shown with FK, primary keys with PK. Dashed edges denote a second, distinctly-named relation between the same two tables (User ↔ Incident has two independent relations: reportedBy and assignedTo).

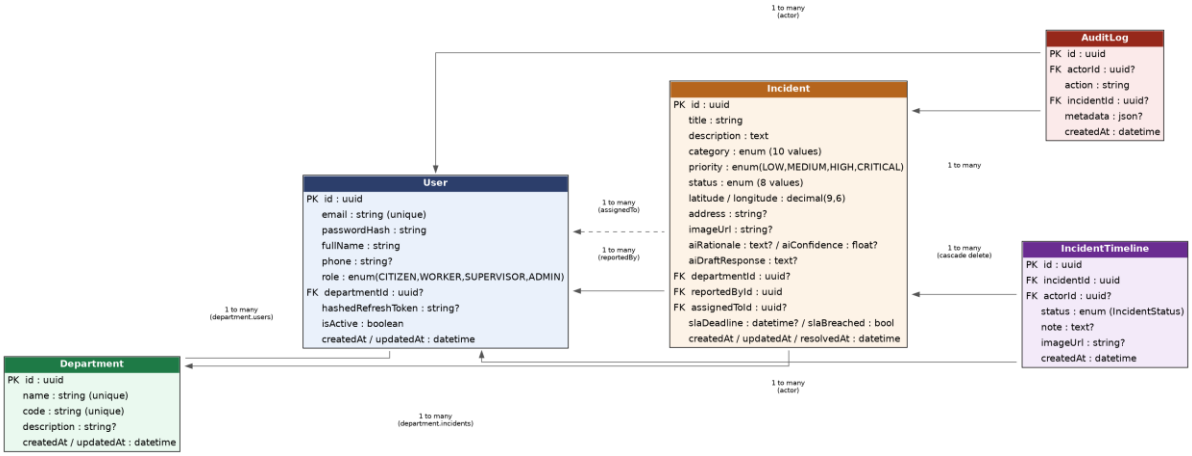

Figure 2.1 — Entity-relationship diagram covering all five database entities and their relationships.

2.1 Entity Descriptions

User

Represents any account in the system regardless of role. A single role enum (CITIZEN, WORKER, SUPERVISOR, ADMIN) determines both frontend routing and backend authorization. departmentId is optional and only meaningful for WORKER and SUPERVISOR accounts, used to scope which incidents a worker can be assigned to.

Department

The routing target for incidents (e.g. Roads, Sanitation, Water). The code field is a free-text string rather than a hardcoded enum specifically so admins can add new departments at runtime via the Admin

Console without a schema migration; the AI classifier's departmentCode output is matched against this field case-insensitively.

Incident

The central entity. Captures the citizen's original submission (title, description, coordinates, optional photo) alongside AI-derived fields (aiRationale, aiConfidence, aiDraftResponse) and SLA tracking fields (slaDeadline, slaBreached) computed by the hourly cron job described in Section 5.

IncidentTimeline

An append-only log of every status transition an incident goes through (SUBMITTED → TRIAGED → ASSIGNED → EN_ROUTE → ON_SITE → RESOLVED → CLOSED, or REJECTED), each optionally attributed to an actor (User) with a note and photo. Cascade-deletes with its parent Incident.

AuditLog

A system-wide audit trail (logins, AI classification decisions, assignments, status changes) queryable by ADMIN for compliance and debugging. actorId and incidentId are both optional since not every logged action is tied to a specific incident (e.g. a login event).

3. API Contract

All endpoints are prefixed with /api/v1. Every route is protected by a global JwtAuthGuard by default; routes explicitly marked Public in the table below opt out via an @Public() decorator. Role restrictions are enforced by a global RolesGuard reading @Roles() metadata. Full interactive documentation is auto-generated at /api/docs via @nestjs/swagger directly from these same decorators and DTOs — it is not hand-written.

3.1 Auth Module

Method	Endpoint	Auth / Roles	Request Body	Response
POST	/auth/register	Public	{ email, password, fullName, phone?, role?, departmentId? }	{ user, tokens: { accessToken, refreshToken } }
POST	/auth/login	Public	{ email, password }	{ user, tokens: { accessToken, refreshToken } }
POST	/auth/refresh	Public (valid refresh token)	{ refreshToken }	{ accessToken, refreshToken } (rotated pair)
POST	/auth/logout	Any authenticated user	(none — uses JWT subject)	204 No Content; hashedRefreshToken cleared

3.2 Users Module

Method	Endpoint	Auth / Roles	Request Body	Response
GET	/users	ADMIN, SUPERVISOR	Query: role?, departmentId?	User[] (id, email, fullName, phone, role, isActive, departmentId, department, createdAt)
PATCH	/users/:id	ADMIN	{ role?, departmentId?, isActive? }	Updated user (id, email, fullName, role, isActive, departmentId)

3.3 Departments Module

Method	Endpoint	Auth / Roles	Request Body	Response
GET	/departments	Any authenticated user	(none)	Department[] ordered by name
POST	/departments	ADMIN	{ name, code, description? }	Created Department
PATCH	/departments/:id	ADMIN	Partial { name?, code?, description? }	Updated Department
DELETE	/departments/:id	ADMIN	(none)	Deleted Department

3.4 Incidents Module

Method	Endpoint	Auth / Roles	Request Body	Response
POST	/incidents	CITIZEN, ADMIN	{ title, description, latitude, longitude, address?, imageUrl?, confirmedNotDuplicate? }	Created Incident, OR { duplicateWarning:true, result } if AI similarity > 0.75 and not yet confirmed
GET	/incidents	Any authenticated (results scoped by role)	Query: status?, priority?, category?, departmentId?, dateFrom?, dateTo?, assignedToId?	Incident[] filtered per query and caller's role
GET	/incidents/:id	Any authenticated user	(none)	Incident with full IncidentTimeline history
PATCH	/incidents/:id/assign	SUPERVISOR, ADMIN	{ workerId }	Updated Incident (status -> ASSIGNED, assignedToId set)
PATCH	/incidents/:id/status	WORKER, SUPERVISOR, ADMIN	{ status, note?, imageUrl? }	Updated Incident + new IncidentTimeline row
POST	/incidents/:id/draft-response	WORKER, SUPERVISOR, ADMIN	{ resolutionNotes }	{ draft: string } — AI-generated citizen-facing message

3.5 AI Proxy Module

Most AI features (classification, duplicate detection, response drafting) are invoked internally by IncidentsService as part of the incident lifecycle and have no standalone endpoint — this is a deliberate design choice, detailed in Section 5, to keep the AI proxy entirely server-side. The Hotspot Predictor is the one AI feature queried directly by the frontend.

Method	Endpoint	Auth / Roles	Request Body	Response
GET	/ai/hotspots	SUPERVISOR, ADMIN	Query: daysBack? (default 90)	{ grid: HotspotCell[], summary: string }

3.6 WebSocket Events (Socket.io, namespace /realtime)

Clients authenticate on connection using the same JWT access token as REST calls (handshake auth.token field). NotificationsGateway auto-joins each socket into rooms — user:<userId>, role:<ROLE>, and dept:<departmentId>:<ROLE> — so events can be targeted precisely without the client subscribing to anything manually.

Event	Emitted When	Payload	Recipients (rooms)
newIncident	A citizen submits a new report	Full Incident object	Workers + supervisors of the routed department; all ADMINs; global broadcast room

Event	Emitted When	Payload	Recipients (rooms)
incidentAssigned	A supervisor assigns an incident to a worker	Updated Incident object	The specific assigned worker (user:<id> room)
incidentUpdated	Status changes, assignment, or resolution	Updated Incident object	Reporting citizen (user:<id>); department supervisors; global broadcast room
slaAlert	Hourly SLA cron detects an at-risk (amber) or breached (red) incident	{ incidentId, level, deadline }	Department supervisors (dept:<id>:SUPERVISOR); all ADMINs

4. React Component Tree

The frontend uses React Router v6 with a single top-level ProtectedRoute wrapper component parameterised by allowedRoles, rather than four separate guard implementations. Global state (authenticated user, incidents collection, live WebSocket listeners) lives in a single Zustand store, useIncidentStore, injected into whichever page components need it rather than passed down via props.

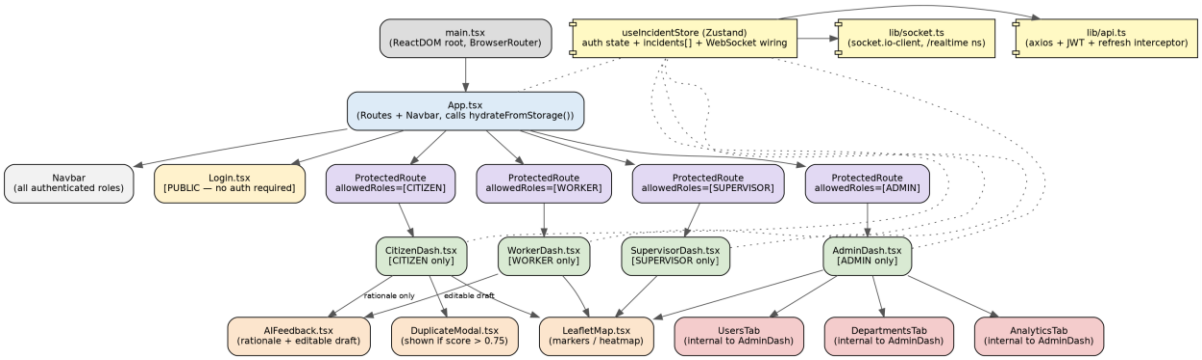

Figure 4.1 — Component tree with role-based route visibility. Dotted lines indicate a store/service dependency rather than a parent-child render relationship.

4.1 Role-Visibility Summary

Route	Allowed Role(s)	Page Component	Key Children
/login	Public (unauthenticated)	Login.tsx	(none — self-contained form)
/citizen	CITIZEN	CitizenDash.tsx	LeafletMap, DuplicateModal, AIFeedback (read-only rationale)
/worker	WORKER	WorkerDash.tsx	LeafletMap, AIFeedback (editable draft)
/supervisor	SUPERVISOR	SupervisorDash.tsx	LeafletMap (with filters + SLA stat cards)
/admin	ADMIN	AdminDash.tsx	UsersTab, DepartmentsTab, AnalyticsTab (LeafletMap + heatmap)

Enforcement is layered: ProtectedRoute redirects an authenticated-but-wrong-role user to their own role's home route (not to an error page), while an unauthenticated user is redirected to /login. This client-side check is a UX convenience only — the authoritative enforcement is the backend's JwtAuthGuard and RolesGuard on every API call, so a manually-edited frontend cannot bypass authorization.

5. AI Integration Architecture

All AI calls are made exclusively from the NestJS backend through a single provider, AiProxyService, using the @google/generative-ai SDK against Google Gemini (model name read from GEMINI_MODEL env var, not hardcoded). No API key or prompt text is ever present in frontend code or network responses sent to the browser — the frontend only ever calls ordinary authenticated REST endpoints and receives already-processed JSON.

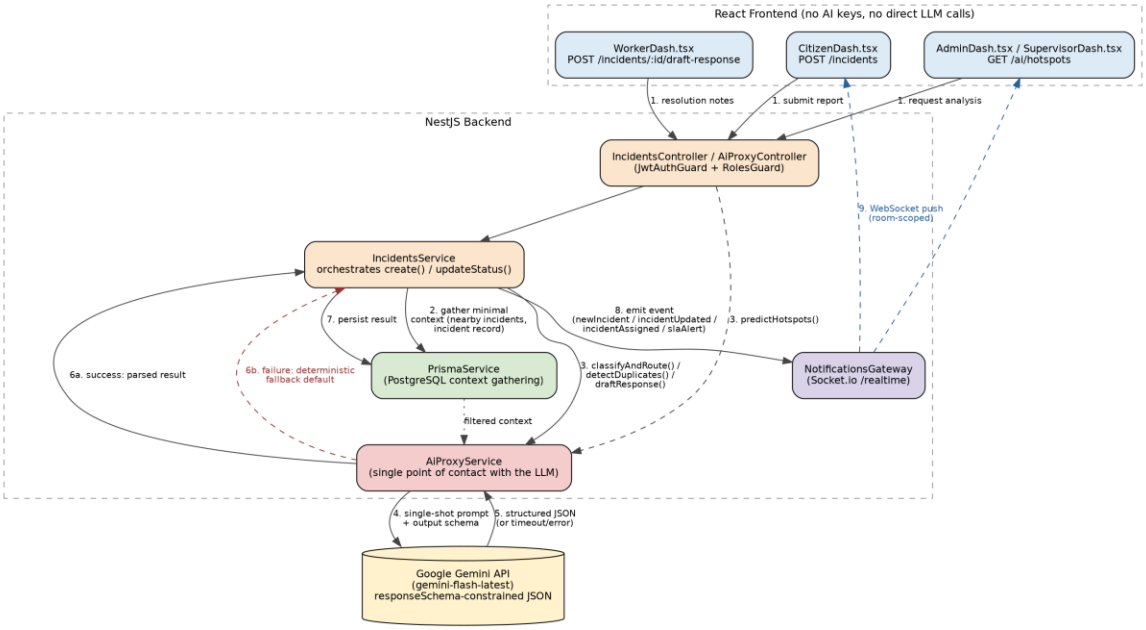

Figure 5.1 — Request flow for an AI-assisted incident submission, from citizen input through classification/duplicate-check to real-time broadcast.

5.1 Prompt Structure

Every prompt follows the same three-part structure: (1) a role-setting instruction establishing the assistant's task ("You are a municipal incident triage assistant..."), (2) the interpolated request-specific data (title, description, coordinates, or resolution notes), and (3) explicit output constraints, enforced structurally via Gemini's responseSchema rather than relying on the model to follow textual formatting instructions alone. This structural enforcement (SchemaType.OBJECT with a required[] field list and enum-constrained fields like category and priority) is what allows downstream NestJS code to consume AI output without defensive parsing. Full verbatim prompt text for all four features is documented in Deliverable 4 (AI Integration Report).

5.2 Context Management

No conversational history or session state is maintained between calls — every AiProxyService method is a single stateless request/response cycle, deliberately, so that token cost and latency scale with a single incident's data rather than an ever-growing conversation. Context is assembled per call by first reducing scope in application code before it ever reaches the LLM:

- Classifier: only the new report's own title/description/address/coordinates.
- Duplicate detector: a two-stage filter (SQL bounding-box query, then exact Haversine distance) narrows the candidate set to at most 20 incidents within the 50-metre radius before any text is sent to Gemini.
- Response drafter: only the single incident's own title, category, and the worker's own resolution notes — no other citizens' data is ever in scope.
- Hotspot predictor: raw incident rows are aggregated into grid cells in JavaScript first; Gemini only ever sees five summarised numeric rows, never raw per-incident free text.

5.3 Output Handling

Each AiProxyService method wraps its Gemini call in a try/catch. On success, result.response.text() (guaranteed valid JSON by responseSchema) is parsed and returned typed via TypeScript interfaces shared between methods (ClassificationResult, DuplicateDetectionResult, DraftResponseResult, HotspotPredictionResult). On failure — network error, quota exceeded, or any thrown exception — every method returns a deterministic, non-AI fallback value rather than propagating the error to the citizen: a safe default classification routed to a manual-triage queue, a proximity-only duplicate match, a template-generated resolution message, or (for hotspots) an unaffected grid with a static summary string. This satisfies the course requirement that AI features degrade gracefully — in this implementation, all four features do so independently, not just the minimum required one.

Once returned to IncidentsService, structured AI output is persisted via Prisma alongside the incident record (aiRationale, aiConfidence, aiDraftResponse columns) so supervisors can review and override the AI's decision at any time, and is simultaneously broadcast over the relevant WebSocket rooms so connected clients see the result without polling.

End of Deliverable 1 — System Design Document.